

Introduction to Machine Learning

Lecture 15: Neural Networks III

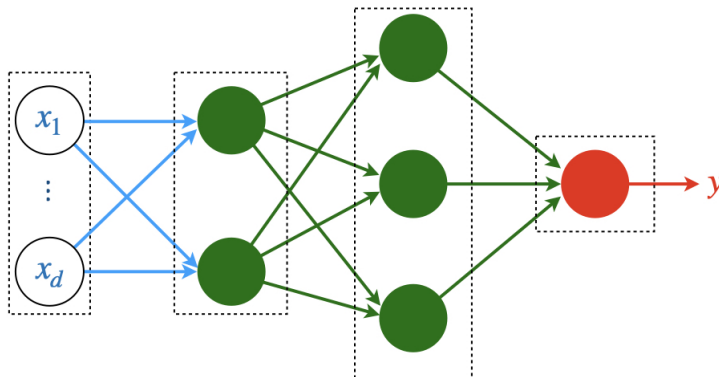
Ali Bereyhi

`ali.bereyhi@utoronto.ca`

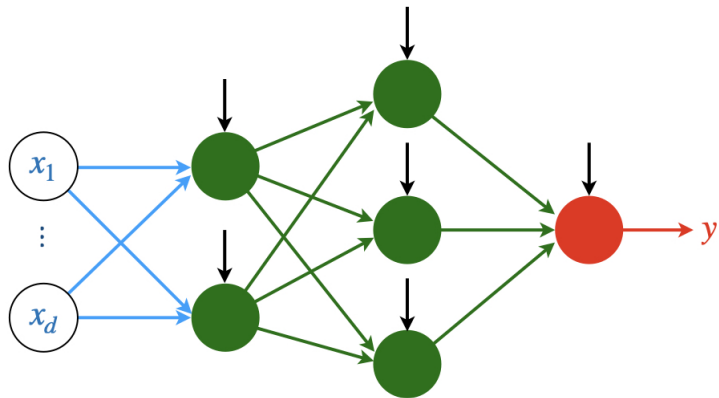
Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

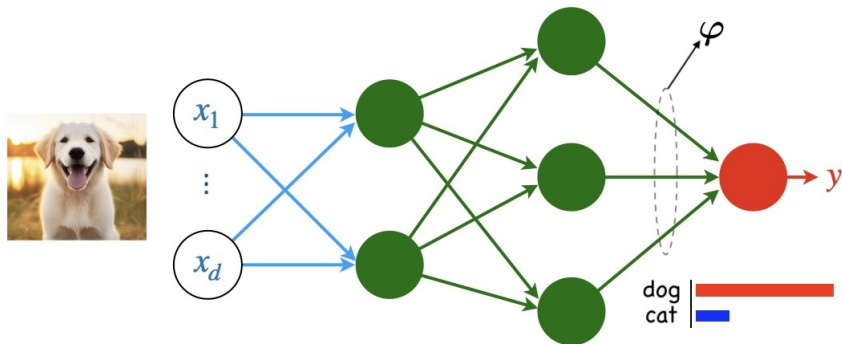
Recap: NN in a Box



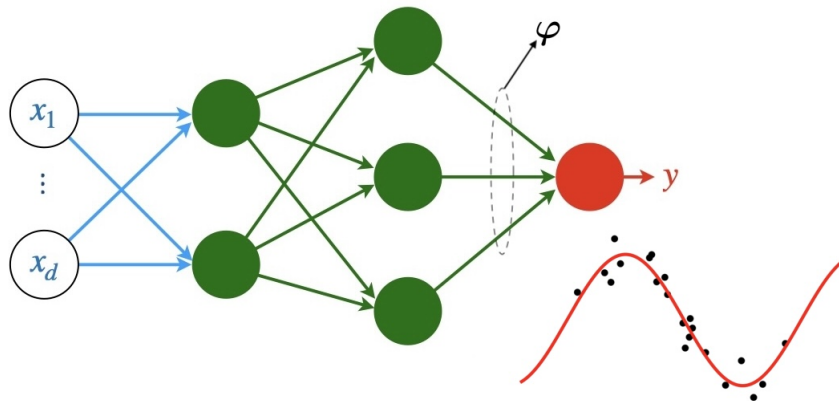
Recap: NN in a Box



Recap: Classification via NNs



Recap: Regression via NNs



Today's Agenda: *Training and Inference via NNs*

Today, we learn

How to Train Neural Networks

In this way, we discuss the following topics

- *Backpropagation*
- *Complete training and test loop*

Recall: Empirical Risk Minimization

Empirical Risk

Say $y_n = f_{\mathbf{w}}(x_n)$ for every $(x_n, \mathbf{v}_n) \in \mathbb{D}$; then, the empirical risk is

$$\hat{R}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}_n, \mathbf{v}_n) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_{\mathbf{w}}(x_n), \mathbf{v}_n)$$

Training is carried out by minimizing the empirical risk

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^m} \hat{R}(\mathbf{w})$$

Recall: *Gradient Descent*

GradientDescent() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}(\mathbf{w}^t)$
- 6: Update $t \leftarrow t + 1$
- 7: **end while**
- 8: Return final $\mathbf{w}^t$

So, we need to find a way to compute $\nabla \hat{R}(\mathbf{w})$ for a given $\mathbf{w}$

Gradient of Empirical Risk

Let's define $\hat{R}_n(\mathbf{w}) = \mathcal{L}(f_{\mathbf{w}}(\mathbf{x}_n), \mathbf{v}_n)$

this is a *sample* loss, i.e., the loss computed for single sample $\mathbf{x}_n$

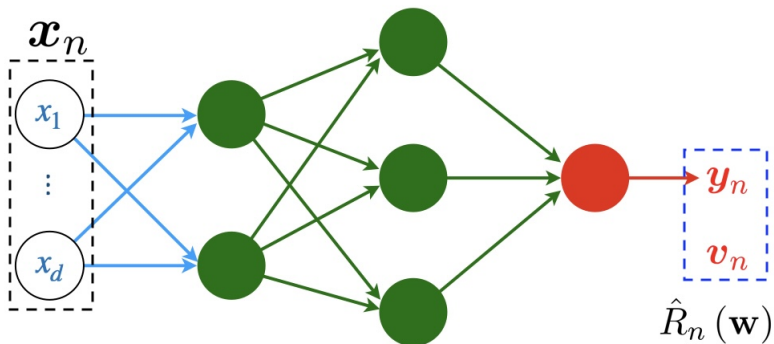
We can write

$$\begin{aligned}\nabla \hat{R}(\mathbf{w}) &= \nabla \frac{1}{N} \sum_{n=1}^N \hat{R}_n(\mathbf{w}) \\ &= \frac{1}{N} \sum_{n=1}^N \nabla \hat{R}_n(\mathbf{w})\end{aligned}$$

Conclusion

We need to find out a way to compute gradient of a sample loss

Gradient of Empirical Risk $\equiv$ Average of Sample Gradients



Computing Sample Gradients

The sample loss is a *nested function* of model parameters

$$\nabla \hat{R}_n(\mathbf{w}) = \nabla \mathcal{L}(f_{\mathbf{w}}(\mathbf{x}_n), \mathbf{v}_n)$$

We need to compute the gradient of a nested function

Nested Function

Let $y = f(z)$ and $z = g(x)$; then, y is a nested function of x

Example: Say $y = z^2$ and $z = 3x$; then,

$$y = z^2 = (3x)^2 = 9x^2$$

Review: Chain Rule

Problem

$y = f(z_1, \dots, z_\ell)$ with $z_i = g_i(x)$: what is derivative of y with respect to x ?

If we change x with a tiny change dx , each z_i changes with

$$dz_i = g'_i(x) dx$$

The changes $dz_1, \dots, dz_\ell$ will change y by

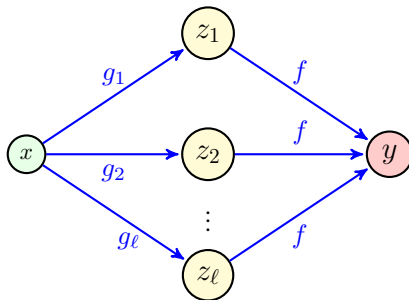
$$dy = \frac{\partial y}{\partial z_1} dz_1 + \dots + \frac{\partial y}{\partial z_\ell} dz_\ell = \frac{\partial y}{\partial z_1} g'_1(x) dx + \dots + \frac{\partial y}{\partial z_\ell} g'_\ell(x) dx$$

So, we have

$$\frac{dy}{dx} = \frac{\partial y}{\partial z_1} \frac{dz_1}{dx} + \dots + \frac{\partial y}{\partial z_\ell} \frac{dz_\ell}{dx}$$

Review: Vectorized Chain Rule

We can show this on a *computation graph*



How Compute Derivatives on Computer: *Example*

Consider the following example

$$z_1 = x^2 \quad z_2 = x^3 \quad y = 2z_1^3 + z_2^2$$

We look for derivative of y w.r.t. x at $x = 1$

Our analytic brain does the following

$$y = 2z_1^3 + z_2^2 = 2(x^2)^3 + (x^3)^2 = 3x^6$$

Then, we would say

$$\frac{dy}{dx}|_{x=1} = 18x^5|_{x=1} = 18$$

How Compute Derivatives on Computer: Example

Consider the following example

$$z_1 = x^2 \quad z_2 = x^3 \quad y = 2z_1^3 + z_2^2$$

We look for derivative of y w.r.t. x at $x = 1$

The computer can only follow the computation graph

```
>> z1 = x**2          z2 = x**3
>> y = 2*(z1**3) + (z2**2)
>> dy_dz1 = 6*(z1**2)      dy_dz2 = 2*z2
>> dz1_dx = 2*x           dz2_dx = 3*x
>> dy_dx = dy_dz1 * dz1_dx + dy_dz2 * dz2_dx
```

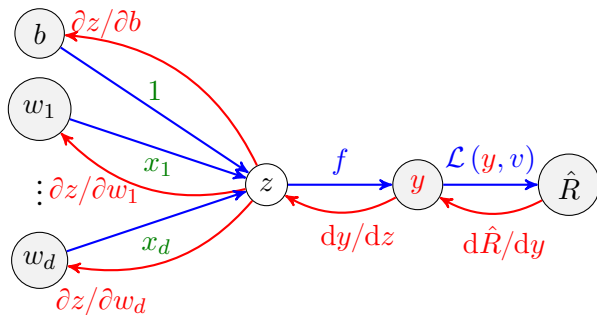
Back to Computing Sample Gradients

The sample loss is a *nested function* of model parameters

$$\nabla \hat{R}_n(\mathbf{w}) = \nabla \mathcal{L}(f_{\mathbf{w}}(\mathbf{x}_n), \mathbf{v}_n)$$

- ? *How can we compute the derivative w.r.t. weights?*
- ! *We represent the NN by a computation graph and use chain rule*

Example: Single Neuron

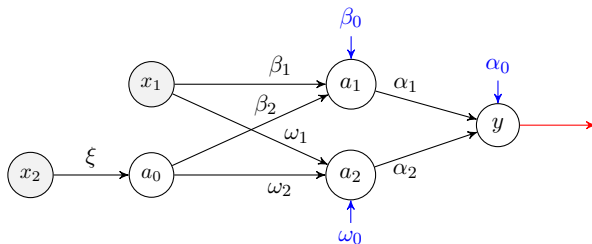


For weight w_n , we can write

$$\frac{\partial \hat{R}}{\partial w_n} = \frac{d\hat{R}}{dy} \frac{dy}{dz} \frac{\partial z}{\partial w_n}$$

the same is valid for the bias

Example: Simple Neural Network



In first hidden node, we have $h_0 = \xi x_2$ and $a_0 = f(h_0)$; then, we get

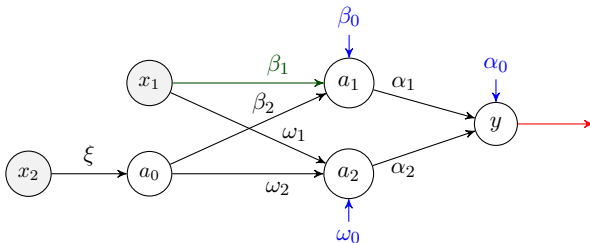
$$h_1 = \beta_0 + \beta_1 x_1 + \beta_2 a_0 \rightsquigarrow a_1 = f(h_1)$$

$$h_2 = \omega_0 + \omega_1 x_1 + \omega_2 a_0 \rightsquigarrow a_2 = f(h_2)$$

and at the output we have

$$z = \alpha_0 + \alpha_1 a_1 + \alpha_2 a_2 \rightsquigarrow y = f(z)$$

Example: Simple Neural Network



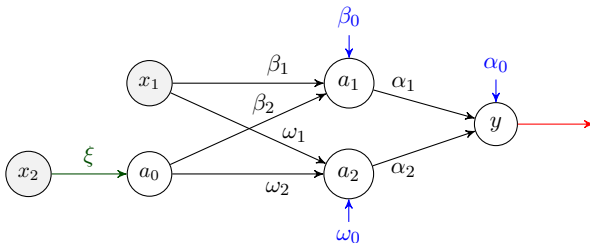
By chain rule, we have

$$\frac{\partial \hat{R}}{\partial \beta_1} = \frac{\partial a_1}{\partial \beta_1} \frac{\partial \hat{R}}{\partial a_1}$$

and we have

$$\frac{\partial \hat{R}}{\partial a_1} = \frac{\partial y}{\partial a_1} \frac{\partial \hat{R}}{\partial y}$$

Example: Simple Neural Network



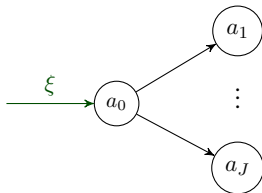
By chain rule, we have

$$\frac{\partial \hat{R}}{\partial \xi} = \frac{\partial a_0}{\partial \xi} \frac{\partial \hat{R}}{\partial a_0}$$

and we have

$$\frac{\partial \hat{R}}{\partial a_0} = \frac{\partial a_1}{\partial a_0} \frac{\partial \hat{R}}{\partial a_1} + \frac{\partial a_2}{\partial a_0} \frac{\partial \hat{R}}{\partial a_2}$$

Extension to General NN



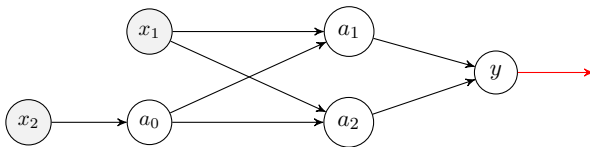
For a neuron with J children, we can write

$$\frac{\partial \hat{R}}{\partial \xi} = \frac{\partial a_0}{\partial \xi} \frac{\partial \hat{R}}{\partial a_0}$$

and we have

$$\frac{\partial \hat{R}}{\partial a_0} = \sum_{j=1}^J \frac{\partial a_j}{\partial a_0} \frac{\partial \hat{R}}{\partial a_j}$$

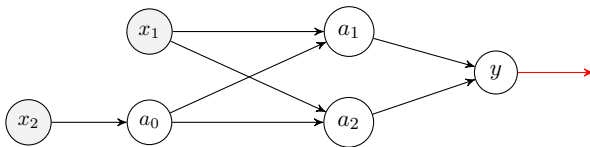
Backpropagation: *Forward Pass*



Forward($x_n | \mathbf{w}^t$):

- 1: Send x_n over the network
- 2: **for** each neuron from *input to output* **do**
- 3: Compute the linear output h and activated output a
- 4: **end for**
- 5: Return all linear and activated outputs

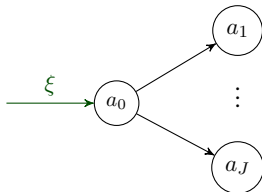
Backpropagation: *Backward Pass*



Backward($\mathbf{v}_n | \mathbf{w}^t$):

- 1: Set $\hat{R}_n = \mathcal{L}(\mathbf{y}_n, \mathbf{v}_n)$ and compute $\nabla_{\mathbf{y}} \hat{R}_n$
- 2: Send $\nabla_{\mathbf{y}} \hat{R}_n$ **backward** over the network
- 3: **for** each neuron a_i from **output to input** **do**
- 4: Compute $\partial \hat{R}_n / \partial a_i$ using its children
- 5: Compute derivative of $\hat{R}_n$ w.r.t. weights of a_i from $\partial \hat{R}_n / \partial a_i$
- 6: **end for**
- 7: Return $\nabla_{\mathbf{w}^t} \hat{R}_n$

Backpropagation: *Need for Gradient*

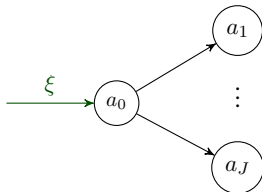


At any activated neuron, we have $a_j = f(h_j)$

$$\begin{aligned}\frac{\partial a_j}{\partial a_0} &= \frac{\partial a_j}{\partial h_j} \frac{\partial h_j}{\partial a_0} \\ &= f'(h_j) \frac{\partial h_j}{\partial a_0}\end{aligned}$$

Activation should be differentiable!

Backpropagation: *Need for Gradient*



By backtracking, we will end up with

$$\frac{\partial \hat{R}}{\partial \xi} \rightsquigarrow \frac{\partial \hat{R}}{\partial a_0} \rightsquigarrow \frac{\partial \hat{R}}{\partial a_j} \rightsquigarrow \dots \rightsquigarrow \nabla_{\mathbf{y}} \hat{R}$$

Loss should be differentiable!

Backpropagation in Practice: *PyTorch Example*

Most packages have backpropagation implemented

```
prediction = model(data) # forward pass
```

We just need to pass backward

```
loss = (prediction - labels).sum()  
loss.backward() # backward pass
```

Further Read

- Goodfellow
 - ↳ Chapter 6: *Section 6.5*
- Bishop
 - ↳ Chapter 4
- ESL
 - ↳ Chapter 7

Backpropagation

Backpropagation

Backpropagation

Gradient Descent with Backpropagation

GradientDescent() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}(\mathbf{w}^t)$
- 6: Update $t \leftarrow t + 1$
- 7: **end while**
- 8: Return final $\mathbf{w}^t$

We know that

$$\nabla \hat{R}(\mathbf{w}^t) = \frac{1}{N} \sum_{n=1}^N \nabla \hat{R}_n(\mathbf{w}^t)$$

We can compute $\nabla \hat{R}_n(\mathbf{w}^t)$ for each n by backpropagation

Gradient Descent with Backpropagation

GradientDescent() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: **for all samples in** $\mathbb{D}$ **do**
- 6: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 7: **end for**
- 8: $\nabla \hat{R}(\mathbf{w}^t) = \text{average} \{ \nabla \hat{R}_1, \dots, \nabla \hat{R}_N \}$
- 9: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}(\mathbf{w}^t)$
- 10: Update $t \leftarrow t + 1$
- 11: **end while**
- 12: Return final $\mathbf{w}^t$

Training with Complete Dataset

FullBatchGD():

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: **for** *all samples in* $\mathbb{D}$ **do**
- 6: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 7: **end for**
- 8: $\nabla \hat{R}(\mathbf{w}^t) = \text{average} \{ \nabla \hat{R}_1, \dots, \nabla \hat{R}_N \}$
- 9: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}(\mathbf{w}^t)$
- 10: Update $t \leftarrow t + 1$
- 11: **end while**
- 12: Return final $\mathbf{w}^t$

Here, we use all samples for every single GD iteration

↳ *We need many GD iterations to converge!*

GD with Single Sample Gradient

SampleGD() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 6: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}_n(\mathbf{w}^t)$
- 7: Update $t \leftarrow t + 1$
- 8: **end while**
- 9: Return final $\mathbf{w}^t$

The idea is very simple

do not loop over samples: update after each backpropagation

GD with Single Sample Gradient: *Possibility of Repetition*

SampleGD() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 6: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}_n(\mathbf{w}^t)$
- 7: Update $t \leftarrow t + 1$
- 8: **end while**
- 9: Return final $\mathbf{w}^t$

? What if after last sample, we end up with initial weights?!

! Well, we could try *shuffling* the samples *randomly*

Stochastic GD

SGD() :

- 1: Choose some *learning rate* η
- 2: Start with an arbitrary $\mathbf{w}^0$
- 3: **while** *not converged* **do**
- 4: Compute the gradient at $\mathbf{w}^t$
- 5: **if** *we finished with all samples in $\mathbb{D}$* **then**
- 6: Shuffle $\mathbb{D}$ *randomly* and start over
- 7: **end if**
- 8: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 9: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}_n(\mathbf{w}^t)$
- 10: Update $t \leftarrow t + 1$
- 11: **end while**
- 12: Return final $\mathbf{w}^t$

Full-Batch Training vs SGD

? Do we pay any price for the speed boost in SGD?!

! Yes! Our gradient estimation have larger **variance**

Gradient Estimate Variance

In fact, we could show that the gradients

$$\nabla \hat{R}(\mathbf{w}^t) \text{ and } \nabla \hat{R}_n(\mathbf{w}^t)$$

are both estimating $\nabla R(\mathbf{w}^t)$, i.e., gradient of the true risk. In this case, the estimators used by SGD are noisier than those used by full batch GD!

Moral of Story

SGD has lower quality of update in each GD iteration

SGD with Mini-Batches

`miniBatchSGD()` :

- 1: Choose some *learning rate* η
- 2: Split $\mathbb{D}$ to mini-batches, each of size B
- 3: Start with an arbitrary $\mathbf{w}^0$
- 4: **while** *not converged* **do**
- 5: Compute the gradient at $\mathbf{w}^t$
- 6: **if** *we finished with all mini-batches* **then**
- 7: Shuffle them *randomly* and start over
- 8: **end if**
- 9: **for** *all samples in the mini-batch* **do**
- 10: Compute $\nabla \hat{R}_n(\mathbf{w}^t)$ with *backpropagation*
- 11: **end for**
- 12: $\nabla \hat{R}_b(\mathbf{w}^t) = \text{average} \{ \nabla \hat{R}_1, \dots, \nabla \hat{R}_B \}$
- 13: Update $\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - \eta \nabla \hat{R}_b(\mathbf{w}^t)$
- 14: Update $t \leftarrow t + 1$
- 15: **end while**
- 16: Return final $\mathbf{w}^t$

Few Definitions

Iteration

Iteration refers to one GD iteration, i.e., after each mini-batch

Epoch

Each time we go through all samples in $\mathbb{D}$, we are finished with one epoch

Batch-size

The size of each mini-batch B is called batch-size

Example

Consider MNIST with 60K samples used with batch-size $B = 100$

Classic Trade-off: *Variance vs Speed*

Mini-batch SGD is the most common form of GD we use

- It provides a *fair* trade-off between speed and variance
- If we set mini-batch size B *large*
 - ↳ We have *less variance* $\rightsquigarrow$ better gradient quality in each iteration
 - ↳ We have *lower speed* $\rightsquigarrow$ longer training time
- If we set mini-batch size B *small*
 - ↳ We have *higher variance* $\rightsquigarrow$ lower gradient quality in each iteration
 - ↳ We have *higher speed* $\rightsquigarrow$ shorter training time

Speed-Variance Trade-off

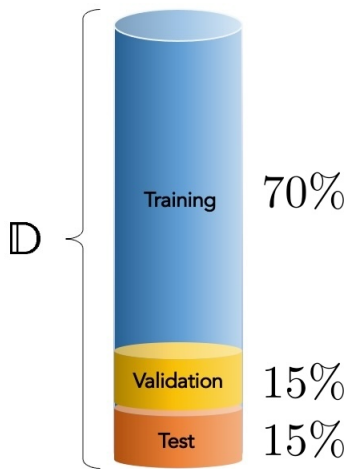
Choice of batch-size specifies the trade-off between variance and speed

Gradient-based Optimizer

Mini-batch SGD is probably not the one you use in your implementation

- *In practice we use other tricks to improve quality of gradient estimation*
 - ↳ *Momentum, learning rate scheduling, . . .*
- *The available optimizers all follow the mini-batch SGD approach*
 - ↳ *They only use tricks to improve quality of $\nabla \hat{R}_b(\mathbf{w}^t)$ before each iteration*
- *Some famous optimizers are*
 - ↳ *Rprop, RMSprop and Adam*
 - ↳ *Check PyTorch for more choices!*

Splitting Data



Building and Training NN

TrainingLoop():

- 1: Build NN $y = f_w(x)$ with some **hyperparameters** and **initial weights**
- 2: Split training set to mini-batches
- 3: Specify the loss function $\mathcal{L}$
- 4: **for** $epochs = 1, \dots, E$ **do**
- 5: Keep applying mini-batch SGD
- 6: **end for**
- 7: Return final weights w^* , average training loss, and accuracy on training set

Validation

Validation():

- 1: Realize NN $\mathbf{y} = f_{\mathbf{w}^*}(\mathbf{x})$ with *trained weights $\mathbf{w}^*$*
- 2: **for** sample i in validation set **do**
- 3: Compute $\mathbf{y}_i = f_{\mathbf{w}^*}(\mathbf{x}_i)$
- 4: Compute sample loss $\hat{R}_i = \mathcal{L}(\mathbf{y}_i, \mathbf{v}_i)$
- 5: Check whether inferred label $\hat{v}_i$ is the same as true label $\mathbf{v}_i$
- 6: **end for**
- 7: Return average validation loss and accuracy

We repeat training with *new hyperparameters* till it's tuned

Evaluation

Evaluation():

- 1: Realize NN $\mathbf{y} = f_{\mathbf{w}^*}(\mathbf{x})$ with *trained weights $\mathbf{w}^*$* and *tuned hyperparameters*
- 2: **for** sample i in evaluation set **do**
- 3: Compute $\mathbf{y}_i = f_{\mathbf{w}^*}(\mathbf{x}_i)$
- 4: Compute sample loss $\hat{R}_i = \mathcal{L}(\mathbf{y}_i, \mathbf{v}_i)$
- 5: Check whether inferred label $\hat{v}_i$ is the same as true label $\mathbf{v}_i$
- 6: **end for**
- 7: Return average evaluation loss and accuracy

Further Read

- Goodfellow
 - ↳ Chapter 6: *Section 6.2 – 6.3*
- Bishop
 - ↳ Chapter 5
- ESL
 - ↳ Chapter 10: *Section 10*

Training

Optimizers

Optimizers